Agentic AI Bootcamp

Research Workflows & Code Auditing

Erkmen G. Aslim & Emily Beam

Department of Economics, University of Vermont

Session 2 · April 27, 2026 · Old Mill A500

eabeam.github.io/uvmecon-ai

What We'll Cover Today

Emily: Research Workflows

- Data exploration pipeline
- Live demo: Lead in Vermont
- Code auditing a replication package
- Paper auditing with `/econ-audit`
- AI vs. real referee reports

Erkmen: Data & Grading

- Web scraping with AI
- Automated grading feedback
- Hands-on practice time

From Chat to Workflows

Session 1 Recap

- Chat AI vs. agentic AI
- The context window
- Voice files & standing instructions
- Skills: reusable workflows

Today's theme:

Multi-step pipelines where AI does the mechanical parts and **you check each step**.

Describe → Propose → Choose → Execute → Review.

You stay in the driver's seat.

Demo 1

Lead Contamination in Vermont

From Curiosity to Map in 20 Minutes

The Question

- Nick Saunders gave a talk on lead contamination
- I wanted to know: **what does lead look like in Vermont?**
- Downloaded TRI data from the EPA — Toxics Release Inventory, public data — filtered to Vermont and lead
- One CSV file. That's it.

This started from **genuine curiosity**, not a prepared exercise.

The Data

- 429 records
- 36 facilities
- 1987–2024 (38 years)
- 24 variables
- Unit: pounds of lead released

The Workflow: Describe, Propose, Choose, Code, Review

Describe the data → Propose ideas → YOU choose → Generate code → Review output

= AI does the mechanical work
= You provide judgment

Each step is **narrow enough to check**. You're not asking the AI to "do my research."

LIVE DEMO

Step 1: Describe the Data

```
$ claude

Read vt_lead_tri.csv and give me a thorough description of
what's in it.  What variables are there, what's the coverage,
who are the top facilities, and anything else I should know
before analyzing it.
```

Switching to terminal...

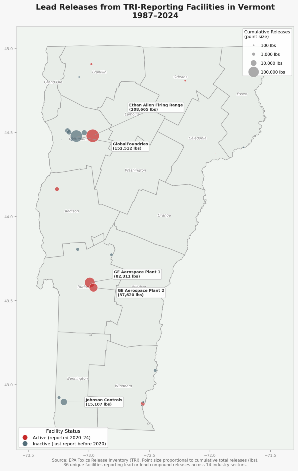
LIVE DEMO

Step 2: Propose Analysis Ideas

```
Based on this data, propose 4 research or visualization ideas
I could pursue, ranging from a quick afternoon project to a
real research project.  For each one, tell me what it would
show, why it's interesting, what additional data I'd need,
and how complex it would be.
```

→ **Key line:** YOU choose what's interesting, not the AI. It generates options. You apply judgment.

The Map: Vermont Lead Releases by Facility



Each dot is a facility. Size = cumulative lead releases (lbs), 1987–2024. Source: EPA TRI.

Key Findings: Two Facilities Dominate

Facility	Lbs
Ethan Allen Firing Range	208,665
GlobalFoundries (Essex Jct)	152,512
GE Aerospace Plant 1 (Rutland)	82,311
GE Aerospace Plant 2	37,620
Johnson Controls (Bennington)	15,107

Top 2 facilities = **69%** of all lead releases.
Top 3 = **85%**.
Total across all facilities: 521,410 lbs.

The firing range is a National Guard site — lead from spent ammunition deposited directly into soil. A completely different pollution story than semiconductor manufacturing.

Time Trend

Before 2007: industrial off-site transfers.
After 2007: military on-site deposition.
The *nature* of pollution changed.

From Curiosity to Written Summary: ∼20 Minutes

The Summary Memo

AI generated a full memo covering:

- Key findings and top facilities
- Time trends and compositional shifts
- Geographic concentration (2 counties = 95% of releases)
- A real limitations section

"TRI is self-reported, doesn't measure actual exposure, only covers facilities above reporting thresholds."

The full pipeline:

1. Describe the data
2. Propose analysis ideas
3. I chose the map
4. AI generated Python code
5. Reviewed the output
6. AI wrote the memo

Total time: ∼20 minutes.
By hand: most of a day.

What AI Is Good At Here vs. Not

✔ AI Strengths

- **Data orientation** — reading a CSV, summarizing, catching quirks
- **Code generation** — the map worked on the first try
- **First-draft reporting** — memo structure, limitations section
- **Proposing angles** you might not think of

✖ Still Needs You

- **Vermont-specific knowledge** — Jericho is near residential areas
- **Policy context** — who uses this, what decisions it informs
- **Deciding what matters** — 4 ideas, can't tell you which matters for *your* work

The AI is a very fast, very thorough RA who **just arrived in Vermont last week**. It can do anything you ask, but it doesn't know what to ask.

Demo 2

Code Audit & Paper Audit

Bangladesh Remote Education RCT

The Problem

- Paper on remote education in Bangladesh
- Evaluates TV instruction, adaptive edtech, data subsidies, and teacher support during COVID-era school closures
- **Keeps getting rejected** — 8 submissions, 11 referee reports
- Used AI two ways:
  - Clean up the code
  - Stress-test the paper

Two use cases:

1. Code audit
Restructure 3 years of accumulated code into a clean replication package

2. Paper audit
Adversarial review before resubmission — catch what referees would catch

Code Audit: Before

```
02_Do files/
01_master_5th June 2021.do
03_vargen_07sep.do
03_vargen.do
05_endline balance_eb.do
05_endline balance.do
09_attrition_8 July_e.do
09_attrition_8 July.do
...
Archive/ (18 files)
WB Submission Files/ (14 copies)
```

The problems:

- **51 do-files** across 4 subdirectories
- Master file from **June 2021** — 3 years stale
- Date-in-filename: `_07sep`, `_02July1130am`
- Author-in-filename: `_eb`, `_e`
- Hardcoded paths — only runs on my machine
- No documentation, no version control

This is **extremely common** in economics.

Code Audit: After

Dimension	Before	After
Do-files	51	**8**
Master file	2021	Current
Config system	None	Template
Version control	None	Git
Documentation	None	Full
"Which version?"	Constant	**Zero**

86 old variants preserved in Archive/ — nothing deleted, fully reversible.

```
bd-remote-ed/
code/00_master_repo.do
code/07_Dofiles_academic/
_globals.do, 02_datagen.do,
03_vargen.do (THE one), ...
Archive/ (86 old)
config/config_local_template.do
docs/   justfile   README.md
```

Time: ~2 Claude Code sessions (a few hours). **By hand:** days, minimum.

The Bug: A Copy-Paste Typo Hiding in Plain Sight

The `$flags` variable list

`$flags` = 17 missing-value indicators used as controls in **every regression**. Defined independently in 5 scripts.

3 of 5 scripts (wrong):

`... _f_c_engaged _f_c_health _c_child_work`

Missing the `_f_` prefix

→

Correct:

`... _f_c_engaged _f_c_health _f_c_child_work`

Result: `_c_child_work` appeared *twice* in regressions. The actual missing indicator was left out. Survived 10+ versions across 2+ years. No error, no crash — just quietly wrong. **How AI found it:** cross-referencing every `global flags` definition across all 51 files.

Paper Audit: What Is `/econ-audit`?

Adversarial Review Skill

A reusable skill that reads a paper and acts as a **skeptical referee**.

- Reads full paper (LaTeX, tables, appendix)
- Checks identification, power, measurement
- Produces structured audit report
- Takes ∼60 seconds

```
$ npx skills add eabeam/econ-skills \
-skill econ-audit
```

What it checks:

1. **Identification:** Is the causal claim valid?
2. **Statistics:** MHT, power, attrition
3. **Measurement:** Are variables measured well?
4. **Methodology:** Design consistency
5. **Presentation:** Framing, length

LIVE DEMO

Running `/econ-audit` on the Paper

```
$ claude
/econ-audit main.tex
```

Reads the full paper + tables + appendix.

Produces a structured audit in ∼60 seconds.

▶ Switching to terminal. . .

AI Audit vs. Real Referees: 7 of 10 in ∼60 Seconds

Referee Criticism	AI?	Notes
High attrition & differential attrition	✔	Flagged as Critical; suggested Lee bounds
Results don't tell a coherent story	◐	Caught ID problem, missed internal contradictions
Theoretical model not useful	✔	Flagged — but recommended *including* it
Short intervention duration (4–8 weeks)	✘	Requires field knowledge
Noisy learning measures (8-item phone test)	✔	Strong match; flagged IRT minimums
Paper too long and hard to follow	✔	Suggested cutting 20–30%
Multiple hypothesis testing issues	✔	Aggressive; added winner's curse framing
Limited external validity / COVID fatigue	◐	Partial; missed publication landscape
Deviations from pre-analysis plan	✔	Caught the spirit, not the specifics
Complex randomization hard to interpret	✔	Strong match

5 full matches, 2 partial, 1 opposite recommendation, 2 misses.
11 referee reports across 8 journals over 4 years — vs. 60 seconds of AI.

What the AI Missed — And Why It Matters

✕ The Misses

- **Short duration:** 4–8 weeks is unusually short for an education RCT. Requires knowing field norms.
- **Internal contradictions:** Data arm increased tutoring *more* than info arm, but only info improved learning. Referees caught this; AI didn't.
- **COVID fatigue:** "We've seen too many of these papers." AI can't know the publication landscape.

The opposite advice:

The AI recommended *including* the conceptual model. Every referee wanted it **cut**.

Sometimes AI gives exactly the wrong recommendation because it lacks field conventions.

Pattern:

AI catches **statistical/econometric** issues reliably.

AI misses **field knowledge**, **within-paper logic checks**, and **publication strategy**.

Takeaway: A Very Good Pre-Submission Check

✓ What It Replaces

- The first 80% of what a careful internal seminar discussant would say
- Catches the systematic stuff: identification, power, MHT, attrition
- In one minute, not one month

What it doesn't replace:

- Field-specific intuition
- "Does this make sense given what we know about Bangladesh?"
- Publication strategy advice
- Colleagues who know your work

For the code audit: it found bugs I didn't find in two years. That alone justified the time.

For the paper audit: run it before you send a paper out. It's free. It catches the embarrassing stuff.

What to Try First

Start Small, Check Each Step

What to Try First

#	Activity	Time	What You Get
1	Create a voice file	15 min	Your tone in every AI draft — reuse forever
2	Data pipeline (describe → propose → code)	30 min	Orientation on any dataset
3	Install `/econ-audit`, run on a paper	10 min	Pre-submission check for free
4	Try inbox triage with Gmail integration	30 min	Sorted, triaged, draft replies

All four use the same principle: **AI does the mechanical parts, you check each step.** Start with whichever one matches a task you already have.

Resources

Links

- **Bootcamp website:**
  `eabeam.github.io/uvmecon-ai`
- **Skills library:**
  `skills.sh`
- **Econ skills:**
  `github.com/eabeam/econ-skills`
- **Claude Code docs:**
  `docs.anthropic.com/en/docs/claude-code`

Install Commands

Step 1: Install Node.js

`nodejs.org` → click LTS

Step 2: Install Claude Code

`npm install -g @anthropic-ai/claude-code`

Step 3: Install econ skills

`npx skills add eabeam/econ-skills`

Step 4: Launch

`claude`

Thank You!

Now over to Erkmen

Erkmen G. Aslim & Emily Beam

Department of Economics, University of Vermont

Bootcamp materials & resources:

eabeam.github.io/uvmecon-ai